

7. Singleton Design Pattern in Java

7.1 Introduction

The **Singleton Pattern** ensures that a class has only one instance and provides a global point of access to it.

7.2 Problem it Solves

When only one instance of a class is needed to coordinate actions across a system (e.g., configuration manager, logging, caching), creating multiple instances could result in inconsistency, resource wastage, or conflicts.

7.3 Key Concepts

- **Private constructor:** Restricts external instantiation
- **Static instance:** Holds the single instance
- **Public accessor method:** Provides access to the instance

7.4 Real-World Analogies

- **Configuration Manager:** You only want one source of truth for application config.
- **Printer Spooler:** Prevent multiple accesses that could result in corruption.

7.5 Variants of Singleton

Example1: Eager Initialization

```
package com.mannemlokesk;  
  
class EagerSingleton {  
    private static final EagerSingleton INSTANCE = new EagerSingleton();  
  
    private EagerSingleton() {  
    }  
  
    public static EagerSingleton getInstance() {  
        return INSTANCE;  
    }  
}  
  
public class Example1EagerTest {  
    public static void main(String[] args) {  
        EagerSingleton obj = EagerSingleton.getInstance();  
        System.out.println("Eager Singleton: " + obj);  
    }  
}
```

Output:

```
Eager Singleton: com.mannemlokesk.EagerSingleton@4617c264
```

Example2: Lazy Initialization (Thread-Unsafe)

```
package com.mannemlokesesh;

class LazySingleton {
    private static LazySingleton instance;

    private LazySingleton() {
    }

    public static LazySingleton getInstance() {
        if (instance == null) {
            instance = new LazySingleton();
        }
        return instance;
    }
}

public class Example2LazyTest {
    public static void main(String[] args) {
        LazySingleton obj = LazySingleton.getInstance();
        System.out.println("Eager Singleton: " + obj);
    }
}
```

Output:

```
Eager Singleton: com.mannemlokesesh.LazySingleton@4617c264
```

Example3: Lazy Initialization with Thread-Safe Block (Improved)

```
package com.mannemlokesesh;

class SynchronizedBlockSingleton {
    private static volatile SynchronizedBlockSingleton instance;

    private SynchronizedBlockSingleton() {
    }

    public static SynchronizedBlockSingleton getInstance() {
        if (instance == null) {
            synchronized (SynchronizedBlockSingleton.class) {
                if (instance == null) {
                    instance = new SynchronizedBlockSingleton();
                }
            }
        }
        return instance;
    }
}

public class Example3SyncBlockTest {
    public static void main(String[] args) {
        SynchronizedBlockSingleton obj =
            SynchronizedBlockSingleton.getInstance();
        System.out.println("Sync Block Singleton: " + obj);
    }
}
```

Output:

```
Sync Block Singleton: com.mannemlokesh.SynchronizedBlockSingleton@4617c264
```

Example4: Bill Pugh Singleton (Best Practice)

```
package com.mannemlokesh;

class BillPughSingleton {
    private BillPughSingleton() {
    }

    private static class HelperHolder {
        private static final BillPughSingleton INSTANCE =
            new BillPughSingleton();
    }

    public static BillPughSingleton getInstance() {
        return HelperHolder.INSTANCE;
    }
}

public class Example4BillPughTest {
    public static void main(String[] args) {
        BillPughSingleton obj = BillPughSingleton.getInstance();
        System.out.println("Bill Pugh Singleton: " + obj);
    }
}
```

Output:

```
Bill Pugh Singleton: com.mannemlokesh.BillPughSingleton@36baf30c
```

Example5: Enum Singleton (Serialization Safe)

```
package com.mannemlokesh;

enum EnumSingleton {
    INSTANCE;

    public void doSomething() {
        System.out.println("Doing work...");
    }
}

public class Example5EnumTest {
    public static void main(String[] args) {
        EnumSingleton.INSTANCE.doSomething();
        System.out.println("Enum Singleton: " + EnumSingleton.INSTANCE);
    }
}
```

Output:

```
Doing work...
Enum Singleton: INSTANCE
```

7.6 Pros and Cons



Pros:

- Controlled access to the sole instance
- Reduces memory footprint
- Useful in multi-threaded logging/config/cache classes



Cons:

- Difficult to unit test (hard to mock)
- Can introduce hidden dependencies
- Thread-safety must be manually handled

7.7 Summary

Variant	Thread Safe	Lazy Loaded	Serialization Safe
Eager Initialization			(default behavior)
Lazy Initialization			
Synchronized Block			
Double-Checked Locking			
Bill Pugh			
Enum Singleton			

The Singleton pattern is simple yet powerful, but must be applied carefully to avoid concurrency issues or design inflexibility.